A Homomorphic Operator Layer for arithmetic that generalizes

Kronecker Embeddings V2 · individual submission · Kunal Sinha

Abstract

We consider embeddings that store mathematical structure — so that the vector of 9 combined with the vector of 9 becomes the vector of 18, and of 81. Storing arithmetic this way is the easy half: a spectral (Fourier) number code turns addition and multiplication into a single fixed operation (elementwise complex product) that is exact and needs zero parameters. The mechanism is not new — it is frequency-domain holographic binding (HRR/VSA; Plate 1995, Kanerva 2009), with the log-domain for multiplication (the classical log-number system), and Fourier number features are known to emerge in trained networks (grokking; Nanda et al. 2023). Our contribution is a **framing and a controlled result**: cast the construction as a *mathematical intermediate representation (IR)* with **explicit operators**, and ask whether making the operation an *architectural primitive* — rather than hoping a network learns it from features — is what enables **magnitude extrapolation**. In a controlled test (same representation, same inputs, same answer-only supervision, varying only the operator), a model with the operator built in extrapolates to unseen magnitudes while the identical model with a learned head instead does not, and a NAC/NALU baseline (Trask et al. 2018) learns the operation only approximately. We then characterise, via representation theory, *which factors* admit such an operator. The evidence supports "the operator, not the representation, is what generalizes" as a tested claim in this controlled setting, not as a universal law.

1. Motivation: a mathematical IR

The usual framing asks whether an embedding can itself perform mathematics (`embedding → answer`). A compiler analogy suggests a stronger one. A compiler never treats `9 + 9` as raw bits from the start; it keeps progressively lower-level representations that *preserve semantics*. At the intermediate-representation (IR) level, `ADD(9, 9)` is not two bit-strings — it carries the meaning "integer addition of these two values," and only later is that lowered to machine ops and bits. Crucially, ADD there is an *explicit operator*, not something inferred from the operands. This reframes the goal: build a **mathematical IR inside a neural representation** — a latent number representation plus explicit operators — so computation becomes compositional in that latent space (`M(9) ⊙ M(9) → M(18)` , then `M(18) ⊙ M(9) → M(162)`). The pipeline is exactly a compiler's: natural language → neural encoder → mathematical IR → transformations → result. More precisely, the tightest correspondence is a compiler's *front-end → IR → constant-folding* path: the number codes and explicit operators are the IR (`M(9) ⊙ M(9) → M(18)` , analogous to `%c = add i32 9, 9`), and evaluating the operator is exactly constant folding (`%c = 18`). The Operator Layer is thus an IR-level *evaluator* rather than a code generator — the natural home for exact arithmetic in representation space, and in sharp contrast to `embedding → answer` , which has no IR at all. The remainder of this report constructs that IR and its operators, and tests the pipeline.

2. Spectral number codes

A number `n` is represented by a bank of complex phasors:

- additive code `a(n) = [e^{i ω_k n}]_k` — for `+`, `−`
- logarithmic code `m(n) = [e^{i ν_k ln n}]_k` — for `×`, `÷`

Both are deterministic and invertible: an integer is recovered by a matched filter that scores candidate values against the code's phases. Two homomorphisms follow directly from the phase, because shifting a number rotates every phasor and a logarithm turns products into sums:

```
a(x + y) = a(x) ⊙ a(y)          m(x · y) = m(x) ⊙ m(y)
```

where `⊙` is the elementwise complex product. Subtraction and division use the conjugate. The operation is therefore a fixed, parameter-free **bind**, and the "embedding of 9" combined with the embedding of 9" literally decodes to 18 (additive band) and 81 (log band).

Why Fourier (a unit-circle Z-transform). The additive code evaluates the number on the **unit circle**, `z = e^{iω}`; the log code is its multiplicative analogue (a Mellin / log-Fourier transform, `n^{iω}`). The Z-transform is the general family — it evaluates at any complex `z = r·e^{iω}` — and the homomorphism `z^{a+b} = z^a·z^b` holds anywhere in the plane. But off the unit circle (`r ≠ 1`) the magnitude `r^n` grows or decays exponentially with `n`, which is numerically unstable and un-decodable. The unit-circle (`r = 1`) restriction keeps every component at magnitude 1 while preserving the exact "add → phase-rotation" homomorphism. It is also canonical, not arbitrary: the characters of the group `(ℤ, +)` live exactly on the unit circle (its Pontryagin dual), so Fourier is *the* basis in which addition diagonalizes — the general point of §6.

Exactness. With no trainable parameters, addition and subtraction decode exactly across the tested range (`a,b < 4000`), multiplication and division decode exactly across the tested product range, and chained additions remain exact to at least 32 terms with no drift.

3. The problem: structure that a model won't use

Exact storage does not imply a model can compute with it and extrapolate. If the spectral features of `a` and `b` are handed to a standard multilayer network that must produce `a + b`, the network fits the training range but collapses on larger, unseen magnitudes. This is the core difficulty: the structure is present in the representation, but a generic learned readout does not inherit it.

4. The Homomorphic Operator Layer

The layer separates what should be learned from what is fixed:

1. **Encode.** Numbers become spectral codes; the words of the query become ordinary learned token embeddings.
2. **Gate.** A small learned gate maps the *phrasing* of the query ("plus"/"add"/"sum" → `+`) to one of the operations. This is the only trained component, and it generalizes across phrasings.
3. **Bind and decode.** The selected exact bind combines the two number-codes and a fixed decoder reads out the integer. This step has no parameters and is valid at any magnitude.

How a query becomes an answer. Two mechanisms are cleanly separated — *which operation* is learned from language, while the *operation itself* is a fixed homomorphism. A reviewer's natural question, "how does the model know to add, and how does it check against `Rep(18)`?", is answered stage by stage:

```
"9 plus 9"
   │  gate:  the word "plus" → operation ADD        (the only learned step)
   ▼
Rep(9), Rep(9)                                      (deterministic encode)
   │  bind:  Rep(9) ⊙ Rep(9) = Rep(18)              (fixed homomorphism, 0 params)
   ▼
Rep(18)
   │  decode: matched filter vs the codebook {Rep(n)} → argmax = 18
   ▼
   18
```

The model never infers "add" from the operands and never *searches* for the answer: the bind *produces* `Rep(18)` by construction, and the only comparison to `Rep(18)` is the decode — a nearest-representation readout that turns the vector back into the integer.

The network thus learns only *what the query asks for*; it never learns arithmetic, which is fixed and exact. That is precisely what converts "memorize a range" into "compute for any magnitude."

5. Controlled experiment: is it the operator or the representation?

A weak comparison would pit the operator layer against an arbitrary baseline and declare victory; that proves little. The claim we actually want to test is causal — *does making the operation an architectural primitive, rather than a learned map, cause magnitude extrapolation?* — so the experiment holds **everything else fixed** and varies only that (`experiments/end2end.py`).

Setup. Queries `"<a> <word> <b>"` over `+` and `−` (kept to two operations so a *differentiable* soft matched-filter decode over a small answer range is tractable); numbers `0..199`; train on pairs with `max(a,b) < 120`, test on `max(a,b) ≥ 120`. Crucially, **supervision is the answer only** — no operation labels — so any op-selection must be learned from the answer signal, and every model sees the same inputs (operand codes + an op-word embedding), the same output space, and the same loss. Three seeds.

model (answer-only supervision)	train	unseen magnitude
HOL — fixed operator (complex-product bind)	1.00	**1.00**
learned bind, shared decode — clean ablation	1.00	**0.00**
learned answer-classifier — weaker ablation	1.00	0.00
NAC / NALU (Trask et al. 2018) on scalar values	0.13	0.05 (within ±1: 0.10)

The **clean ablation** is the load-bearing comparison. It keeps the *same* operand codes and the *same* differentiable matched-filter decode over the *same* candidate codebook, and replaces **only** the fixed complex-product bind with a learned map on the same codes. Because the decode ranges over the whole codebook, unseen-magnitude answers stay fully reachable — so its failure is not a label-coverage artifact. It fits training (1.00) and still

collapses out of distribution (0.00), while the fixed operator holds at 1.00. That isolates the cause: the *exactness of the operator*, not the representation or the reachable output set, is what extrapolates. (The weaker "answer-classifier" ablation reaches the same 0.00 but partly for a structural reason — its fixed answer-class head never sees test-range labels in training — so we do not lean on it; it is listed only for completeness.)

Two honest caveats. First, HOL's *only* learned parameters are the small op-word gate, so its train/extrapolation are near-trivial to fit (std 0 across seeds) — the point is not that HOL learns something hard, but that its fixed operator carries exactness to any magnitude where the learned bind cannot. Second, the NAC baseline is not a strawman: it genuinely learns the operation (weights converge to ≈ `[1, 1]` for `+` and `[1, −1]` for `−`), but NAC weights only *approach* integer values, so its error is multiplicative and grows with magnitude (mean absolute error `2.1 → 4.3`), which exact-integer scoring penalises — precisely the value of an *exact* operator over a learned-approximate one.

This supports — in this controlled setting — that arithmetic extrapolation here is **architectural**: it comes from the operator, not from the features. (The earlier `operator_layer.py` covers all four operations `+ − × ÷` with an op-word gate and is kept as a breadth illustration; the causal claim rests on the clean ablation above.)

6. What are the right factors? A factorization ablation

The structured-token idea `byte ⊗ position` beats an arbitrary learned token ID because it factors identity into meaningful parts. For numbers the sharper question is not "can we copy that?" but *what are the right factors?* Different factorizations serve different jobs: `byte ⊗ position` is right for identity, `digit ⊗ place` for decimal notation and magnitude, and a **character / irreducible-representation basis** (Fourier `e^{iω n}` for `+`, log-Fourier for `×`) for the *operation* — because in that basis the operation becomes pointwise. The unifying principle is representation-theoretic: the right factors are the characters of the algebraic structure whose operation you want exact and cheap, i.e. the basis in which that operation diagonalizes.

We exhibit this directly (`experiments/factorization.py`) by factoring an integer three ways and giving each the operator natural to it — this measures operator *admissibility*, not a learned-model comparison (that is §5). Each entry is the exact accuracy of that factorization's own operator across magnitude bands (n = 60–120 random pairs per band):

factorization	natural operator	exact across magnitudes?	admits an exact operator?
Fourier characters `e^{iω n}`	pointwise complex product (0 params)	100% (bands to 2×10^5, n=60–120)	yes — pointwise, bounded range
`digit ⊗ place` (base 10)	digit-wise add + carry	100%, unbounded	yes — sequential (carry)
arbitrary learned ID	(none; a learned MLP must memorise)	train 100%, unseen 0%	no

Two conclusions follow. First, **a factorization is useful for an operation iff it admits a simple exact operator, whatever its ability to represent the number**: all three can represent any integer, yet only the two with an exact operator compute it across magnitudes; the arbitrary learned ID has *no* operator, so a model can only memorise and then fails on unseen inputs (its 0% is a memorisation failure; with a fixed answer-class head, test-range answers are also unseen labels — the clean learned comparison is the §5 ablation). This is the Kronecker lesson

sharpened — structure helps because it *admits an operator*. Second, among structured factors there is a **range ↔ operation tradeoff**: the character basis diagonalizes addition and yields a single pointwise operator, but has a finite unambiguous range; `digit ⊗ place` does not diagonalize addition and needs sequential carries, but is exact and unbounded. Because a ring has no single basis diagonalizing both `+` and `×` (the reason the main method uses two bands), a joint representation or a cheap change-of-basis between operation-bases is the deeper open direction.

7. Discussion

The number code is appended alongside a model's existing word or byte dimensions, matching the intended "keep the existing dimensions for language, add mathematics in new ones" design. Several directions extend the result: (i) making the decode step differentiable end-to-end so the gate can be trained purely from answer supervision; (ii) handling magnitude beyond a fixed decode window, where the log code's monotonic phase provides a natural, order-preserving signal; and (iii) growing the fixed, shallow AST (a single binary operation) into **nested/parenthesised expressions** — since each operator returns a valid IR code, results compose directly (`M(18) ⊙ M(9) → M(162)`), so a parser feeding a tree of binds is the natural path — and rendering the final result back to natural language to close the NL→NL loop. Everything reported here is deterministic, self-contained, and reproducible on CPU in about a minute.

Limitations (stated plainly). The controlled result (§5) is scoped to `+` and `−`, with **deterministic operand encoding** — numbers are fed as their spectral codes rather than parsed from raw text — and the learned component is a small op-word gate. So the claim is precisely "given the numbers already encoded, an *operator* built into the architecture extrapolates where a learned head does not," not "an end-to-end model learns to read and compute." The Fourier band also has a finite unambiguous range and an O(magnitude) matched-filter decode; `×` / `÷` were kept out of the differentiable test because their answer range makes a soft decode costly. The mechanism is prior art (HRR/VSA binding, the log-number system, grokking-Fourier); the framing, the controlled ablation, and the "right factors" analysis are the contribution.

The single next step that would make this a paper — now taken (§8). Drop the operator layer, as a *differentiable* module, into a small end-to-end transformer that reads the raw string `"<a> word <b>"` character by character (so operand **parsing is learned**, not given), and compare against (a) the identical parser with the fixed bind replaced by a learned bind and (b) a strong learned baseline that emits the answer digit by digit. Section 8 reports exactly this experiment. Its finding is twofold and honest: learned parsing does **not** bootstrap from the answer signal alone in this small setting (the open, hard part — length/magnitude generalization of learned arithmetic parsing); but **once parsing is supervised, the architectural claim carries over in full** — the fixed operator extrapolates perfectly to unseen magnitudes (1.00) while the identical model with a learned bind over the *same* parsed operands collapses (0.00), and the digit-sequence transformer fits in-range and collapses out (0.02). The remaining gap to a paper is closing the parsing bootstrap (curriculum or auxiliary parse signal) and nested expressions.

8. Learned operand parsing from raw text

Section 5 handed every model the operand codes. The honest objection is that the operands were a given. Here they are not: a small **character-level transformer reads the raw string** (`' 48 plus 79'`, `' 7 minus 153'`) and must itself produce the operand codes; only then does the operator combine them and a shared matched-filter

decoder read the answer (`experiments/parse_end2end.py`). Operands are `0..199` (three digits); train on `max(a,b) < 120`, test on `max(a,b) ≥ 120`; `+` and `−`; three seeds.

How the parser can extrapolate at all — and the tie back to Kronecker. The operand code is assembled by the exact homomorphism $a(d_2\cdot 100 + d_1\cdot 10 + d_0) = a(d_2\cdot 100) \odot a(d_1\cdot 10) \odot a(d_0)$: a product of per-digit, per-place phasor codes. Place is positional — exactly the Kronecker *value ⊗ position* bias. The transformer must learn to read each raw character into a digit; because the per-slot digit reader shares weights across places, "the tens slot is a 7" generalises from `70..79` to `170..179`, so operand codes stay exact at magnitudes never seen in training. At evaluation the digit reads are taken as exact (argmax → integer → exact code), so a correctly-read operand yields an exact answer through the fixed operator.

Finite decode range and cost (explicit). Decoding is a matched filter over a fixed candidate codebook spanning `[−200, 400]`: `NC = 601` candidates, each a $2\cdot K_a$-vector, so decode costs $\approx NC\cdot 2K_a = 76{,}928$ multiply-adds **per example** and the codebook must be widened to admit larger answers. This is a bounded-range evaluator (a constant-folding IR), *not* an unbounded arithmetic unit — `O(magnitude)` by construction.

Two regimes, reported side by side (3 seeds).

model (char transformer reads raw text)	train	unseen magnitude
parse + **fixed operator** — answer-only loss	0.00	0.00
parse + **fixed operator** — with parse supervision	1.00	**1.00**
parse + **learned bind** — with parse supervision (clean ablation)	1.00	**0.00**
digit-sequence decoder — no operator (strong baseline)	1.00	0.02

Two things are true at once, and both are stated plainly. **(1) The hard part is parsing.** With answer-only supervision the reader does not bootstrap in this small setting — it never fits training, let alone extrapolates. This is the known frontier (length/magnitude generalization of learned arithmetic parsing), and we do not claim to have solved it. **(2) Given parsing, the architectural claim carries over in full.** When the reader is taught by a disclosed per-digit cross-entropy — and the answer loss then trains only the *combiner*, so reader and combiner are learned the same way for both arms — the fixed operator extrapolates perfectly to unseen magnitudes (1.00), while the identical model that reads the *same* operands but **learns** the bind fits training and collapses out of distribution (0.00). The learned bind shares the same decode over the same codebook, so test-range answers remain reachable: its collapse is a real failure of the learned operator, not a label-coverage artifact. The digit-sequence transformer — a model that just emits the answer tokens — fits in-range and collapses out (0.02), confirming that extrapolation is not free without the operator.

The experiment therefore both *extends* §5 (the operator-versus-representation result now holds with learned parsing, not only with given codes) and *localises what remains open* (bootstrapping the parse from answer-only supervision). That is the honest state of the work: the operator is established as the source of magnitude generalization; closing the parsing bootstrap, and growing from a single binary operation to nested expressions, is what remains.

9. Multiplication and spelled-number input

Section 8 read digit characters and did `+` / `−`. The natural question — does the layer handle an input like *"forty eight times ninety six"*? — needs two more pieces: English **number words** as operands and the **multiplicative** operator. Both are added here (`experiments/parse_mult.py`). A word-level transformer reads five tokens `[A_tens A_ones "times" B_tens B_ones]` (e.g. `["forty","eight","times","ninety","six"]`, real English, a pad token where a ones-word is absent), parses each operand to its value, and the **fixed multiply operator** — the log-code complex product (`×` → add of log-phases) — binds the two codes; a matched filter decodes the product. The headline case resolves correctly: *forty eight times ninety six* → **4608**.

What this isolates (and how it differs from §8). Section 8 tested whether *reading* generalises across magnitude. Here parsing is deliberately **in-distribution**: every number word `1..99` is seen in training (balanced parse supervision), so the parser is a seen lookup. The extrapolation axis is the **product**: train on pairs with `a·b < 3000`, test on `a·b ≥ 3000` (up to `99·99 = 9801`). This isolates whether the *operator* computes unseen, larger products.

model (reads "forty eight times ninety six")	train	unseen large product
parse + **fixed multiply** (operator)	1.00	**1.00**
parse + **learned bind** (clean ablation, same codebook)	0.57	**0.00**
digit-sequence decoder (no operator, strong baseline)	1.00	0.00

The fixed operator extrapolates perfectly to products it never saw (1.00): the log-code homomorphism makes `×` exact at any magnitude, so once the operands are read the product is exact. The learned bind — reading the *same* operands, decoded over the *same* product codebook so unseen products are reachable — cannot even reliably fit the training products (0.57; the multiplicative bind is a hard function to learn) and extrapolates not at all (0.00). The digit-sequence transformer fits in-range and collapses out (0.00): a model that emits the product digit by digit has no way to reach a magnitude it never saw.

Finite decode range and cost — explicit, and larger than §8. The product codebook spans `[1, 9801]`: `NC = 9801`, so decode costs $\approx NC \cdot 2 \cdot K_m \approx 9.4\times10^5$ multiply-adds per example — about 15× the additive band of §8 — the `O(magnitude)` price of a wider window, stated plainly. One numerical detail matters at this scale: adjacent large products are log-close, so the matched filter is run in double precision (float32 accumulation would occasionally pick a neighbour); with perfect reads the decode is then exact over all 2,869 achievable products.

Scope. Operands are `1..99` (two word-slots) and the operation is a single `×`; the spelled-number parse is a seen-vocabulary lookup (this experiment isolates the operator, not reading extrapolation — that was §8). Together §8 and §9 bracket the claim from both sides: the operator is what generalises, whether the open axis is reading magnitude (§8, additive) or product magnitude (§9, multiplicative).

10. Computational cost and limitations

What kind of arithmetic is this, next to a machine's 32-bit or 64-bit integers? The honest answer rests on one split: the **bind** (the operation) and the **decode** (reading the integer back) have completely different cost profiles, and nearly every property follows from that.

The **bind** — add/subtract as an elementwise complex product of additive codes, multiply/ divide as the same in log-codes — is `O(K)` in the number of frequencies (`K = 48..64` here), exact, parameter-free, and

independent of magnitude: measured, `5+3`, `10^6+10^6` and `10^15+10^15` all cost about 10 µs. It also never overflows, since `a(x) ⊙ a(y) = a(x+y)` holds for any real `x, y` — in that one respect it is more graceful than a fixed-width integer, which wraps at `2^32`. The **decode** is the cost: a matched filter over a candidate window, `O(NC·K)` with `NC` the window size.

Versus hardware integers. A hardware integer stores the value positionally, so the read is free and the whole `[0, 2^32)` / `[0, 2^64)` range comes by construction; the operation is `O(1)` but overflows at the width. Here the representation is `2K` floats (magnitude-independent), the operation is `O(K)` and overflow-free, but the read is a `O(NC·K)` search.

aspect	32/64-bit hardware integer	spectral operator layer
representation size	fixed 32/64 bits — `O(log n)`	`2K` floats, magnitude-independent
the operation (add/mul)	`O(1)`, exact, overflows at `2^32` / `2^64`	`O(K)` complex product, exact, no overflow
reading the value back	free — it *is* the bits	matched-filter search, `O(NC·K)`
range	`[0, 2^32)` / `[0, 2^64)` by construction	finite unambiguous range per frequency set; float-bounded
big numbers	still `O(1)` within the width	decode cost grows with range; precision ceiling

A flat Fourier band is not an efficient route to those ranges: it would need either `O(range)` candidates (≈4×10^9 — infeasible) or a structured factorization (the CRT / digit⊗place routes of §6 and the appendix) giving `O(log range)` frequencies and a hierarchical decode — which reintroduces carries and structure. So this is a bounded-range **evaluator** (a constant-folding IR), not a general ALU.

Measured decode cost (CPU, single-thread, `K = 64`). Decode time is linear in the window:

decode window	time	mul-adds
10^2	0.9 ms	12.8 K
10^3	1.6 ms	128 K
10^4	15 ms	1.28 M
10^5	118 ms	12.8 M
10^6	1.06 s	128 M

So multiplying two ~5-digit numbers (products to ≈10^{10}) by brute-force decode means scanning a 10^{10} window — seconds to minutes, clearly impractical. That is the `O(range)` wall. Storage, by contrast, is flat: a billion and a 5 both take `2K` floats.

Do large operands need a GPU? No. The bind never does. The decode is embarrassingly parallel (a matrix–vector product plus argmax), so a GPU raises *throughput*, but it does not change the `O(range)` scaling. The real lever is the decode *algorithm* — a hierarchical coarse-to-fine or CRT phase-unwrapping decode cuts readout from `O(range)` toward `O(K·log range)` on a plain CPU; this implementation uses the brute-force matched filter for simplicity and exactness checking.

Scaling to large magnitudes. Two constraints bite as the result grows: *time* (the `O(range)` brute-force decode) and *exactness* (float precision). The table extrapolates the measured CPU anchor (10^6 window = 1.06 s, linear) and estimates an optimized GPU decode as a dense matmul (~10^{13} FLOP/s, ~$10^5\times$ this single-core path; optimistic — beyond ~10^{12} the candidate codebook cannot be stored and the decode is memory-bound). The bind is ~10 µs for every row. Crucially, the last two columns show that *exactness*, not time, is usually the binding constraint: an ✗ means the decoded integer is already wrong, whatever the compute budget.

result magnitude	brute-force decode, CPU (1 core)	brute-force decode, good GPU (est.)	exact `+` / `−`	exact `×` / `÷`
10^3	~1 ms	<1 µs	✓	✓
10^6	~1 s	~10 µs	✓	✓
10^9	~18 min	~10 ms	✓	✗
10^{12}	~12 days	~11 s	✓	✗
10^{15}	~34 years	~3 hours	✓ (near 2^{53} edge)	✗
10^{30}	~3×10^{16} years	~3×10^{11} years	✗ (not representable)	✗

So brute-force decode is only sensible to ~10^6–10^9 (ms–minutes on CPU, sub-second on GPU); past that, cost *and* precision rule it out. A structured `O(K·log range)` decoder would be µs–ms at any row, but it does not lift the exactness ceilings — those need higher-precision phases or a residue (CRT) representation, not more compute.

A fast decoder, measured (`experiments/fast_decode.py`). We implemented that structured decoder for the additive band and benchmarked it. A geometric frequency ladder (`ω_j = 2π / radix^j`) is decoded by **coarse-to-fine phase unwrapping**: each frequency gives the residue `n mod λ_j` , and decoding refines coarse → fine, resolving each level's wrap from the running estimate — `O(K)` with `K ≈ log_radix(range)` frequencies and *no* candidate scan. The homomorphism `a(x)⊙a(y)=a(x+y)` holds for any frequency set, so the same exact `+` / `−` bind is unchanged; only the readout changes. Measured (2000 samples per range, all exact):

range	frequencies `K`	hierarchical decode	brute-force decode	speedup
10^3	4	~11 µs	~1.1 ms	~$10^2\times$
10^6	6	~13 µs	~1.1 s	~$8\times10^4\times$
10^9	7	~17 µs	~18 min	~$6\times10^7\times$
10^{12}	9	~18 µs	~12 days	~$6\times10^{10}\times$

Decode time is essentially flat (~10–20 µs) because it is `O(K)` : `123456789 + 987654321` binds and decodes to the exact `1111111110` in ~18 µs on one CPU core. It is exact to 10^{15} and breaks at 10^{16} — confirming both the `2⁵³` additive ceiling and the section's headline: **the fast decoder removes the cost wall but not the precision wall.** (This is an inference-time readout; training still uses the differentiable soft matched filter over a small window.)

The two precision ceilings differ sharply (additive vs multiplicative). The additive phase step is a constant (`ω·1` , magnitude-independent), so `+` / `−` stay exact nearly to `2⁵³ ≈ 9×10¹⁵` in float64. The multiplicative

step shrinks as `1/P` (the information is in log-phase), so `×` / `÷` lose integer resolution far earlier — measured, the log band resolves `P` from `P±1` only to about **10^8** in float64 (and ~10^4 in float32). A product like $10^{15} \times 10^{15} = 10^{30}$ is doubly lost: 10^{30} is not even a distinct float64 integer (`1e30 + 1 == 1e30`), so there is nothing exact to decode to.

Limitations, plainly. (1) Readout is a search, not free — `O(range)` naive, the core inefficiency versus positional binary's `O(1)` read. (2) A fixed frequency set has a finite unambiguous range; hardware-scale ranges need `O(range)` frequencies or CRT/digit structuring. (3) Two different floating-point precision ceilings: `+` / `−` are exact nearly to $2^{53} \approx 9 \times 10^{15}$ (constant phase step), but `×` / `÷` only to about **10^8** in float64 (~10^4 in float32), because the log-phase step shrinks as `1/P` — so large products are lost to precision long before cost, and 10^{30} is not even a representable float64 integer. (4) The strength is narrow and the weakness specific — the operation is exact and overflow-free at any magnitude (`O(K)`); everything hard lives in representation range and decode cost, not in the arithmetic itself.

Appendix: alternatives considered

"Comply with the Kronecker factorization" means finding a representation `entity ⊗ basis` in which the operation acts simply on the factors. Several candidates were weighed before choosing the Fourier/character route; they differ mainly in *which* operation each basis makes componentwise, and in their range.

method	`+`	`×`	**complies with Kronecker how**	**tradeoff**
Residue Number System (CRT)	componentwise $\bmod p_i$	componentwise $\bmod p_i$	the *most literal* — $\mathbb{Z}/M \cong \otimes_i \mathbb{Z}/p_i$ is a ring tensor factorization; both ops componentwise	needs a second per-prime code so `×` is a rotation too (discrete log); **bounded** (mod M)
Polynomial / digit-DFT	coeff. add + carry	convolution → **pointwise in the DFT domain** (convolution theorem; FFT multiplication)	keeps `digit ⊗ place`, and the DFT over *place* makes multiplication pointwise	still needs carry; two-stage
Fourier / character (used here)	phase rotation $e^{i\omega n}$	log-Fourier / Mellin $n^{i\omega}$	operation-diagonalizing basis; each op becomes a pointwise product	needs two bands (`+` and `×` don't share a basis); bounded range
Irreducible-representation / matrix binding	1-D irreps on the unit circle (= Fourier)	—	the general lens: factors are the operation's irreps; non-abelian ops give matrix-valued factors and binding becomes matrix multiplication	heavier machinery; only needed for non-commutative operations

method	`+`	`×`	complies with Kronecker how	tradeoff
Learned bilinear / tensor operator	learn `T` with `Rep(a+b)=T(Rep(a),Rep(b))`	same	a bilinear map is itself a Kronecker-structured 3-tensor; connects to "grokking" (Fourier features emerge)	may not generalize unless the tensor is constrained

The Residue Number System is the tightest single-factorization fit (both operations componentwise), and the digit-DFT is the most elegant route for multiplication while retaining the `digit ⊗ place` structure. This work chooses the Fourier/character basis because it makes each operation an exact, parameter-free *pointwise* product and is the canonical basis in which addition diagonalizes (§2, §6). There is no free lunch: the right factorization is chosen relative to the operation and the numeric range required.

Related work (honest scoping)

The *mechanism* here is not new. Encoding a number as unit phasors and binding by an elementwise complex product is **Holographic Reduced Representations / Vector Symbolic Architectures** (Plate 1995; Kanerva 2009); doing multiplication in the log domain is the classical **log-number system**; and Fourier/"clock" number features are known to *emerge* in trained networks that learn modular arithmetic (**grokking**; Nanda et al. 2023). **NAC/NALU** (Trask et al. 2018) is the standard learned module for arithmetic extrapolation, and is used here as a baseline. What this submission contributes is not the encoding but (i) the **framing** — a mathematical IR with explicit operators, under a compiler analogy; (ii) a **controlled result** that the *operator-as-architecture*, not the features, is what extrapolates (§5); and (iii) the **representation-theoretic "right factors" analysis** (§6). These are claims about framing and evidence, honestly scoped, not a new primitive.

All results are produced by the accompanying code (`experiments/exactness.py` *,* `experiments/end2end.py` *,* `experiments/operator_layer.py` *,* `experiments/factorization.py` *); the interactive companion runs the same computation live in the browser.*